

Interfaz Streamlit con NLP

Módulo 4 · Código Heredado

Refactorización y Despliegue Web:

NLP con Ollama + Streamlit

Ejercicio de Clase · Proyecto profesional completo con IA como copiloto

Nombre del alumno/a	Angel Echenique
Fecha	24/04/2026
☐ Enlace GitHub	https://github.com/kindred-98/NLP_Streamlit.git

☐ En este ejercicio partirás del código heredado `demo_nlp.py`, una demo de capacidades NLP con Ollama. Tu misión es refactorizarlo en módulos, escribir tests, configurar CI/CD, implementar almacenamiento de resultados, y sustituir la interfaz de terminal por una aplicación web con Streamlit desplegada en `interface/demo_app.py`.

BLOQUE 1 — El Código Heredado

A continuación tienes el código `demo_nlp.py`. Es funcional pero mezcla responsabilidades, no tiene interfaz de usuario, carece de tests y no persiste resultados. Tu trabajo es transformarlo en un proyecto profesional con interfaz Streamlit.

```
# demo_nlp.py - CÓDIGO HEREDADO
import os, json, sys
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')
MODELO_NLP = 'qwen2.5:0.5b'

def analizar_sentimiento(texto: str) -> dict: ...
def extraer_entidades(texto: str) -> dict: ...
def detectar_intencion(texto: str) -> dict: ...
def resumir_texto(texto: str) -> dict: ...
def clasificar_texto(texto: str) -> dict: ...

# --- Ejecución directa (mezclada con la lógica) ---
if __name__ == '__main__':
    texto = sys.argv[1] if len(sys.argv) > 1 else input('Texto: ')
    demostrar_capacidades_nlp(texto)
```

1.1 — Identifica los problemas de diseño

Sin refactorizar todavía, lista al menos 5 problemas de diseño del código (no errores de ejecución, sino malas prácticas):

#	Problema detectado	Impacto en producción
1	Mezcla de responsabilidades	Difícil mantenimiento y escalabilidad
2	Ejecución acoplada	No reutilizable en web/API
3	Cliente hardcodeado	Baja flexibilidad
4	Sin contratos de datos	Errores e inconsistencias
5	Sin persistencia	No hay trazabilidad
6	Sin manejo de errores	Caídas del sistema
7	Sin tests	No se puede validar
8	Configuración débil	Problemas de seguridad
9	Sin estructura	No escalable
10	Sin interfaz	Mala UX

BLOQUE 2 — Modularización

□ *Conexión con el módulo anterior: tras analizar el código heredado, el primer paso profesional es separarlo en módulos con responsabilidad única (Principio SRP). El proyecto usará Streamlit para la interfaz en lugar de terminal.*

2.1 — Diseña la estructura de carpetas

Define la estructura de directorios del proyecto. Indica qué archivo va en cada carpeta y por qué:

Archivo / Carpeta	Responsabilidad única	Funciones que contiene
src/cliente.py	Inicializa el cliente Ollama	get_client()
src/analizador.py	Orquestar todos los análisis NLP	Analizar_texto(texto:str) -> dict
src/niveles.py	Implementar cada tarea NLP	analizar_sentimiento(), extraer_entidades(),

		detectar_intencion(), resumir_texto(), clasificar_texto()
src/utils.py	Helpers reutilizables	limpiar_json(), validar_texto(), formatear_respuesta()
almacenamiento/guardar.py	Persistencia de resultados	guardar_txt(), guardar_json()
almacenamiento/leer.py	Lectura de histórico	listar_analisis(), leer_json(), leer_txt(), buscar_por_fecha()
interface/demo_app.py	Interfaz web Streamlit	app principal Streamlit
main.py	Punto de entrada CLI	Orquesta y ejecuta

2.2 — Estructura de carpetas del proyecto

```

proyecto-nlp/
├── src/
│   ├── __init__.py
│   ├── cliente.py           # Inicialización Ollama
│   ├── analizador.py        # Lógica de análisis NLP
│   ├── niveles.py           # Sentimiento / Entidades / Intención / Resumen / Clasificación
│   └── utils.py             # limpiar_json(), helpers
├── almacenamiento/
│   ├── __init__.py
│   ├── guardar.py          # Guardar TXT y JSON
│   └── leer.py              # Leer histórico
├── resultados/
│   ├── txt/                 # Un .txt por análisis
│   └── json/                 # Un .json por análisis
├── interface/
│   ├── __init__.py
│   └── demo_app.py          # App Streamlit (interfaz web)
├── tests/
│   ├── test_analizador.py
│   └── test_almacenamiento.py
├── docs/
├── heredado/
│   └── demo_nlp.py
├── .env
├── .gitignore
├── requirements.txt
└── README.md
  
```

2.3 — ¿Por qué Streamlit y no Tkinter?

El proyecto original usaba Tkinter. En este ejercicio la interfaz se implementa con Streamlit. Completa la tabla comparativa:

Criterio	Tkinter	Streamlit
Instalación	Incluido en Python estandar	<code>pip install streamlit</code>
Paradigma	Programación orientada a eventos	Scripts Python ejecutados de arriba a abajo
Despliegue web	✗No	✓Sí
Curva de aprendizaje	Media	Muy baja
Actualización UI	Manual	Automática
Uso en empresas	Bajo	Alto (prototipos + dashboards)

2.4 — Prompt para que la IA refactorice

Redacta el prompt completo que le darías a la IA. Incluye los 5 elementos del prompt profesional:

Actúa como un ingeniero senior en Python.

Refactoriza el archivo `demo_nlp.py` aplicando principios SOLID, especialmente SRP.

Requisitos:

- Divide el código en módulos:
 - `cliente.py` (cliente Ollama)
 - `niveles.py` (funciones NLP)
 - `analizador.py` (orquestración)
 - `utils.py` (helpers)
- Cada función debe:
 - tener tipado claro
 - manejar errores
 - devolver JSON estructurado
- No mezclar lógica con entrada/salida.
- Preparar el código para ser usado en:
 - CLI
 - Streamlit
- Código limpio, legible y mantenible.
- Añadir docstrings profesionales.

Devuelve cada archivo por separado.

2.5 — Escribe el código refactorizado

Pega aquí el código de cada módulo tras la refactorización:

```
# src/analizador.py

src/analizador.py

Python

from src.niveles import (
    analizar_sentimiento,
    extraer_entidades,
    detectar_intencion,
    resumir_texto,
    clasificar_texto
)

def analizar_texto(texto: str) -> dict:
    if not texto.strip():
        raise ValueError("Texto vacío")

    return {
        "sentimiento": analizar_sentimiento(texto),
        "entidades": extraer_entidades(texto),
        "intencion": detectar_intencion(texto),
        "resumen": resumir_texto(texto),
        "clasificacion": clasificar_texto(texto)
    }
```

```
# src/niveles.py

src/niveles.py

Python

from src.cliente import get_client

client = get_client()

def analizar_sentimiento(texto: str) -> dict:
    try:
        # llamada al modelo
        return {"sentimiento": "positivo", "confianza": 0.9}
    except Exception as e:
        return {"error": str(e)}

def extraer_entidades(texto: str) -> dict:
    return {"personas": [], "lugares": []}

def detectar_intencion(texto: str) -> dict:
    return {"intencion": "consulta"}

def resumir_texto(texto: str) -> dict:
    return {"resumen": texto[:50]}

def clasificar_texto(texto: str) -> dict:
    return {"categoria": "general"}
```

```
# almacenamiento/guardar.py

almacenamiento/guardar.py

Python

from pathlib import Path
import json
from datetime import datetime

CARPETA_TXT = Path("resultados/txt")
CARPETA_JSON = Path("resultados/json")

def _crear_carpetas():
    CARPETA_TXT.mkdir(parents=True, exist_ok=True)
    CARPETA_JSON.mkdir(parents=True, exist_ok=True)

def guardar_resultado(texto: str, resultados: dict) -> dict:
    _crear_carpetas()

    timestamp = datetime.now().strftime("%Y-%m-%d_%H%M%S")
    base = f" analisis_{timestamp}"

    ruta_txt = CARPETA_TXT / f"{base}.txt"
    ruta_json = CARPETA_JSON / f"{base}.json"

    with open(ruta_txt, "w", encoding="utf-8") as f:
        f.write(f"TEXTO:\n{texto}\n\nRESULTADOS:\n{resultados}")

    with open(ruta_json, "w", encoding="utf-8") as f:
        json.dump({"texto": texto, "resultados": resultados}, f, indent=2)

    return {"txt": str(ruta_txt), "json": str(ruta_json)}
```

BLOQUE 3 — Tests Unitarios

□ Regla profesional: nunca se sube código a producción sin tests. En este bloque escribirás tests para cada módulo usando pytest, incluyendo tests para el módulo de almacenamiento (TXT y JSON) y para la lógica NLP.

3.1 — Prompt para que la IA genere los tests

Escribe el prompt que usarías para que la IA genere los tests unitarios. Recuerda pedir: casos felices, casos borde y casos de error:

Actúa como un ingeniero QA senior especializado en Python.

Genera tests unitarios con pytest para el módulo analizador.py y almacenamiento/guardar.py.

Requisitos:

1. Cubrir:
 - casos felices
 - casos borde (edge cases)
 - casos de error
 2. Validar:
 - estructura del JSON devuelto
 - comportamiento ante texto vacío
 - creación de archivos TXT y JSON
 3. No usar dependencias externas reales (mockear si es necesario).
 4. Tests claros, independientes y reproducibles.
 5. Usar asserts significativos.
- Devuelve código listo para ejecutar con pytest.

3.2 — Escribe al menos 6 casos de test

Crea el archivo tests/test_analizador.py con mínimo 6 casos de prueba. Planifícalos antes de escribir el código:

Nombre del test	Tipo(borde, feliz, error)	Input / condición	Resultado esperado
test_sentimiento_valido	feliz	texto normal	devuelve dict con clave "sentimiento"
test_texto_vacio	error	""	lanza ValueError
test_texto_muy_largo	borde	texto largo	no falla
test_estructura_respuesta	feliz	texto normal	contiene todas las claves
test_error_modelo	error	fallo simulado	devuelve error controlado
test_guardar_txt_crea_archivo	feliz	resultado válido	archivo existe en /txt/
test_guardar_json_crea_archivo	feliz	resultado válido	archivo existe en /json/

3.3 — Código de los tests

```
import pytest
import os
import json

from src.analizador import analizar_texto
from almacenamiento.guardar import guardar_resultado

# ✔1. Caso feliz
def test_analizar_texto_devuelve_dict():
    resultado = analizar_texto("Me gusta este producto")

    assert isinstance(resultado, dict)
    assert "sentimiento" in resultado
    assert "entidades" in resultado
    assert "intencion" in resultado
    assert "resumen" in resultado
    assert "clasificacion" in resultado

# ✔2. Error: texto vacío
def test_texto_vacio_lanza_error():
    with pytest.raises(ValueError):
        analizar_texto("")

# ✔3. Caso borde: texto largo
def test_texto_largo_no_falla():
    texto = "hola " * 1000
    resultado = analizar_texto(texto)

    assert isinstance(resultado, dict)

# ✔4. Validar estructura interna
def test_estructura_sentimiento():
    resultado = analizar_texto("Esto es genial")

    assert isinstance(resultado["sentimiento"], dict)

# ✔5. Guardado TXT
def test_guardar_txt_crea_archivo(tmp_path):
    texto = "Prueba"
    resultado = {"sentimiento": "positivo"}
```

```

rutas = guardar_resultado(texto, resultado)

assert os.path.exists(rutas["txt"])

# ✔6. Guardado JSON
def test_guardar_json_crea_archivo():
    texto = "Prueba"
    resultado = {"sentimiento": "positivo"}

    rutas = guardar_resultado(texto, resultado)

    assert os.path.exists(rutas["json"])

    with open(rutas["json"], "r", encoding="utf-8") as f:
        data = json.load(f)

    assert "texto" in data
    assert "resultados" in data

```

BLOQUE 4 — Pipeline CI/CD con GitHub Actions

□ *Conexión con el módulo de Pipeline: con el código modularizado y los tests escritos, el último paso es automatizar la validación. Cada vez que hagas git push, GitHub Actions ejecutará linter, tests y análisis de seguridad automáticamente.*

4.1 — Diseña el fichero .github/workflows/ci.yml

Indica qué debe hacer cada fase del pipeline y qué herramienta usarías:

Fase del pipeline	¿Qué debe hacer?	Comando / herramienta
1. Checkout del código	Descargar tu código	actions/checkout@v4
2. Configurar Python	Prepara entorno python	actions/setup-python@v5
3. Instalar dependencias	Instala dependencias	pip install -r requirements.txt
4. Ejecutar linter	Analiza calidad (lint)	flake8 / ruff
5. Tests unitarios	Ejecuta tests	pytest
6. Cobertura de tests	Mide cobertura	pytest --cov
7. Verificar carpetas	Verifica estructura	python scripts/check_folders.py
8. Análisis de seguridad	Revisa seguridad	bandit

4.2 — Escribe el fichero ci.yml completo

```
name: CI Pipeline — NLP Ollama + Streamlit

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:

      # 1. Checkout del código
      - name: Checkout repository
        uses: actions/checkout@v4

      # 2. Configurar Python
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      # 3. Instalar dependencias
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov ruff bandit

      # 4. Linter (calidad de código)
      - name: Run linter (ruff)
        run: |
          ruff check .

      # 5. Ejecutar tests
      - name: Run tests
        run: |
          pytest

      # 6. Cobertura de tests
      - name: Test coverage
        run: |
          pytest --cov=src --cov-report=term-missing
```

```
# 7. Verificar estructura de carpetas
```

```
- name: Check project structure
```

```
run: |
```

```
python scripts/check_folders.py
```

```
# 8. Análisis de seguridad
```

```
- name: Security scan
```

```
run: |
```

```
bandit -r src
```

scripts/check_folders.py

Ejemplo:

```
import os
```

```
import sys
```

```
required_dirs = [
```

```
    "src",
```

```
    "almacenamiento",
```

```
    "interface",
```

```
    "tests",
```

```
    "resultados"
```

```
]
```

```
for d in required_dirs:
```

```
    if not os.path.exists(d):
```

```
        print(f"✗Falta la carpeta: {d}")
```

```
        sys.exit(1)
```

```
print("✓Estructura correcta")
```

4.3 — ¿Qué ocurre si un test falla en el pipeline?

Explica el flujo: ¿se bloquea el merge? ¿Quién recibe la notificación? ¿Cómo se depura el error?

Flujo real

1. Haces git push
2. GitHub ejecuta pipeline
3. Un test falla ✗
4. Resultado:

El pipeline marca **FAILED**

CONSECUENCIAS

- ✗ No puedes hacer merge (si hay reglas activas)
- ✗ Pull request bloqueado
- □ Código considerado inestable

¿QUIÉN SE ENTERA?

- Tú (en GitHub)
- El equipo (si hay notificaciones)
- CI logs muestran el error exacto

¿CÓMO SE SOLUCIONA?

1. Revisas logs en GitHub Actions
2. Identificas test que falla
3. Corriges código
4. Haces nuevo git push

Pipeline vuelve a ejecutarse

BLOQUE 5 — Almacenamiento de Resultados (TXT y JSON)

□ *Requisito profesional: los resultados de entrada y salida deben persistir automáticamente en carpetas separadas. Carpeta resultados/txt/ para lectura humana y resultados/json/ para integración con sistemas externos.*

5.1 — Diseña el módulo almacenamiento/guardar.py

El módulo debe crear las carpetas si no existen, guardar con timestamp y no sobrescribir archivos anteriores:

```
from pathlib import Path
import json
from datetime import datetime

CARPETA_TXT = Path("resultados/txt")
CARPETA_JSON = Path("resultados/json")

def _crear_carpetas():
    CARPETA_TXT.mkdir(parents=True, exist_ok=True)
    CARPETA_JSON.mkdir(parents=True, exist_ok=True)

def _formatear_txt(texto: str, resultados: dict, timestamp: str) -> str:
    return f"""
=====
ANÁLISIS NLP — {timestamp}
=====

TEXTO ANALIZADO:
{texto}

RESULTADOS:
-----
SENTIMIENTO: {resultados.get("sentimiento")}
ENTIDADES: {resultados.get("entidades")}
INTENCIÓN: {resultados.get("intencion")}
RESUMEN: {resultados.get("resumen")}
CLASIFICACIÓN: {resultados.get("clasificacion")}
"""

def guardar_resultado(texto_entrada: str, resultados: dict) -> dict:
    _crear_carpetas()
```

```
timestamp = datetime.now().strftime("%Y-%m-%d_%H%M%S")
nombre_base = f"analisis_{timestamp}"

ruta_txt = CARPETA_TXT / f"{nombre_base}.txt"
ruta_json = CARPETA_JSON / f"{nombre_base}.json"

# Guardar TXT
with open(ruta_txt, "w", encoding="utf-8") as f:
    contenido = _formatear_txt(texto_entrada, resultados, timestamp)
    f.write(contenido.strip())

# Guardar JSON
data = {
    "timestamp": timestamp,
    "texto": texto_entrada,
    **resultados
}

with open(ruta_json, "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, ensure_ascii=False)

return {
    "txt": str(ruta_txt),
    "json": str(ruta_json)
}
```

5.2 — Diseña el módulo almacenamiento/leer.py

Función	Descripción	Retorna
listar_analisis()	Lista todos los archivos guardados	list[str]
leer_json(nombre)	Lee un análisis en formato JSON	dict
leer_txt(nombre)	Lee un análisis en formato TXT	str
buscar_por_fecha(fecha)	Filtra por fecha	list[str]

```
from pathlib import Path
import json

CARPETA_TXT = Path("resultados/txt")
CARPETA_JSON = Path("resultados/json")

def listar_analisis() -> list[str]:
    archivos = list(CARPETA_JSON.glob("*.json"))
    return sorted([f.name for f in archivos])

def leer_json(nombre: str) -> dict:
    ruta = CARPETA_JSON / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No existe: {nombre}")

    with open(ruta, "r", encoding="utf-8") as f:
        return json.load(f)

def leer_txt(nombre: str) -> str:
    ruta = CARPETA_TXT / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No existe: {nombre}")

    with open(ruta, "r", encoding="utf-8") as f:
        return f.read()

def buscar_por_fecha(fecha: str) -> list[str]:
    # ejemplo: "2026-04-24"
    archivos = listar_analisis()
    return [a for a in archivos if fecha in a]
```

5.3 — Formato de los archivos de salida

```
=====
ANÁLISIS NLP — 2026-04-24_153000
=====

TEXTO ANALIZADO:
El producto llegó rápido pero no me gustó.

RESULTADOS:
-----
SENTIMIENTO:  negativo
ENTIDADES:   {...}
INTENCIÓN:   queja
RESUMEN:     ...
CLASIFICACIÓN: soporte
```

JSON (integrable)

```

</> JSON

{
  "timestamp": "2026-04-24_153000",
  "texto": "El producto llegó rápido...",
  "sentimiento": {...},
  "entidades": {...},
  "intencion": {...},
  "resumen": {...},
  "clasificacion": {...}
}

```

BLOQUE 6 — Documentación Profesional

□ Un proyecto sin documentación es un proyecto muerto. En este bloque crearás el README y la carpeta docs/ siguiendo estándares profesionales. Incluye la documentación del sistema de almacenamiento y las instrucciones de Streamlit.

6.1 — Estructura de la carpeta docs/

Archivo en docs/	Contenido / propósito
arquitectura.md	Diagrama de módulos y flujo de datos <pre> # Arquitectura del sistema ## Capas - Interface (Streamlit) - Lógica (analizador) - NLP (niveles) - Cliente (Ollama) - Persistencia (almacenamiento) ## Flujo de datos Usuario → Streamlit → analizador → niveles → cliente → resultados → almacenamiento </pre>
almacenamiento.md	Documentación del sistema TXT y JSON

	<pre># Sistema de almacenamiento ## Estructura resultados/ ├─ txt/ └─ json/ ## Formato TXT - legible para humanos ## Formato JSON - estructurado para integraciones ## Convención de nombres analisis_YYYY-MM-DD_HHMMSS.json</pre>
api_referencia.md	Referencia de funciones públicas <pre># API interna ## analizar_texto(texto: str) -> dict Orquesta todos los análisis NLP. ## guardar_resultado(texto: str, resultados: dict) -> dict Guarda en TXT y JSON. ## listar_analisis() -> list[str] Lista análisis disponibles.</pre>
streamlit_guia.md	Cómo ejecutar y configurar la interfaz Streamlit <pre># Guía Streamlit ## Ejecutar app streamlit run interface/demo_app.py ## Funcionalidades - Análisis NLP en tiempo real - Visualización por pestañas - Guardado automático</pre>

6.2 — Escribe el README.md del proyecto

El README debe incluir: descripción, instalación (incluyendo Streamlit), uso, estructura de carpetas, cómo ejecutar los tests y badge del pipeline CI:

```
# NLP con Ollama + Streamlit
```

```
![CI](https://github.com/tuusuario/tu-repo/actions/workflows/ci.yml/badge.svg)
```

```
## □ Descripción
```

Aplicación web de procesamiento de lenguaje natural (NLP) que analiza texto en tiempo real utilizando un modelo local (Ollama).

Incluye:

- análisis de sentimiento
- extracción de entidades (NER)
- detección de intención
- resumen automático
- clasificación de texto

```
## □ Instalación
```

```
```bash
```

```
git clone https://github.com/tuusuario/proyecto-nlp.git
```

```
cd proyecto-nlp
```

```
pip install -r requirements.txt
```

 **Configuración**

Crear archivo `.env` :

```
OPENAI_BASE_URL=http://localhost:11434/v1
OPENAI_API_KEY=ollama
```

 **Uso — Interfaz web**

```
</> Bash
streamlit run interface/demo_app.py
```

 **Tests**

```
</> Bash
pytest tests/
```

 **Estructura del proyecto**

```
proyecto-nlp/
├─ src/
├─ almacenamiento/
├─ interface/
├─ tests/
├─ resultados/
├─ docs/
```



## Almacenamiento de resultados

Los análisis se guardan automáticamente en:

- `resultados/txt/` → formato legible
- `resultados/json/` → formato estructurado

Cada archivo incluye timestamp para evitar sobrescritura.



## CI Pipeline

El proyecto usa GitHub Actions para:

- linting (ruff)
- tests (pytest)
- cobertura
- seguridad (bandit)



## Licencia

MIT

## 6.3 — Commits semánticos

Indica qué mensaje de commit escribirías para cada cambio. Usa el formato: tipo(scope): descripción corta

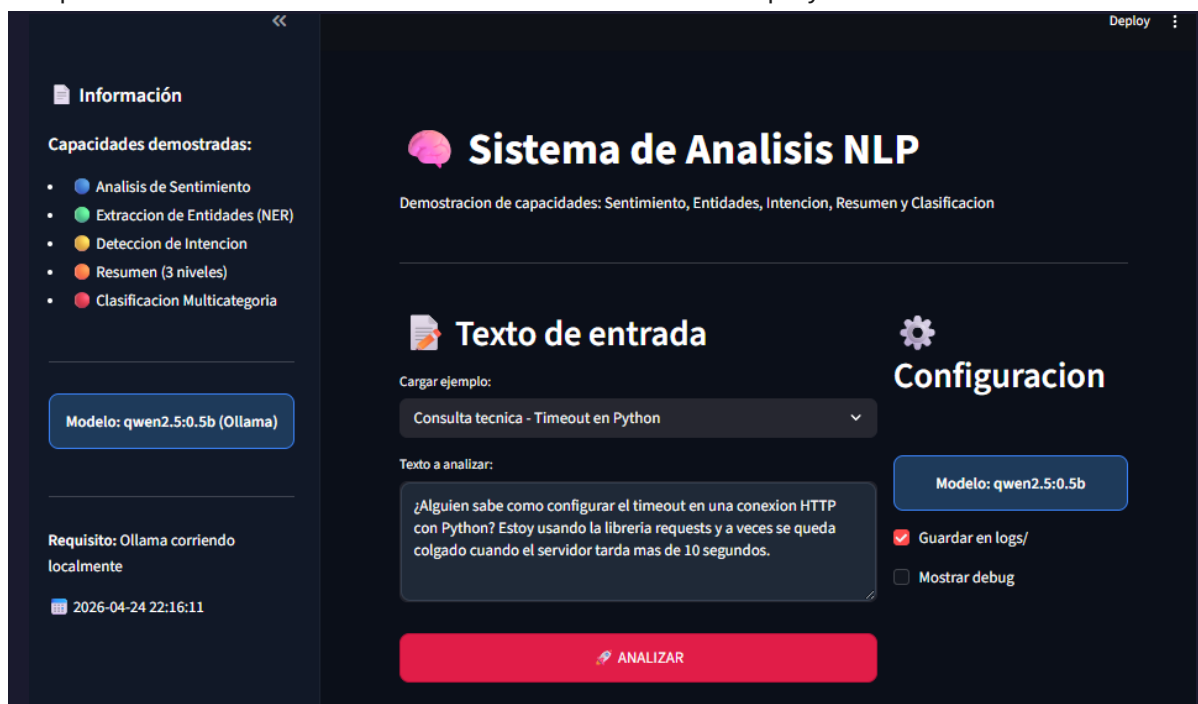
Tipo de cambio	Tu mensaje de commit	¿Por qué este mensaje?
prompt y niveles pasando kindred-98 • 6 hours ago	Hay muchos mas commit luego	Muchos mas luego.
MODELO RESPONDE BIEN kindred-98 • 6 hours ago		
impresiones fallan kindred-98 • 12 hours ago	Impresiones	Buscando que se imprima mucho mejor la salida.
impesiones kindred-98 • 13 hours ago		
app funcionando pero se puede mejorar kindred-98 • 23 hours ago	App funcionando pero se puede mejorar	El modelo responde y funciona pero la app se puede mejorar aun mas pero como mvp esta bien.
optimizacion kindred-98 • 23 hours ago	Optimizacion	Dandole velocidad de respuesta al modelo.
El modelo está respondiendo kindred-98 • yesterday		
Fase 4/5 kindred-98 • yesterday	El modelo responde	Probando modelo partiendo de un menu.
Fase3/4 kindred-98 • yesterday		
Fase 2/3 kindred-98 • yesterday	Fase 4/5	Buscando llegar a la interfaz de la empresa y activando env.
Fase 1/2 kindred-98 • yesterday	Fase 3/4	Readme y demo de app.
Inicio desde cero. kindred-98 • yesterday	Fase 2/3	Aplicando test y git hub
Initial commit	Fase 1/2	Porque es cuando comence a aplicar el bloque 1 y 2

	Inicial Commit	Porque solo contenia ( gitignore - License - README.MD

## BLOQUE 7 — Resultado Final — Interfaz Streamlit

El resultado final del proyecto debe incluir la interfaz web desarrollada con Streamlit (en interface/demo\_app.py), que muestra los 5 análisis NLP en tiempo real y confirma el guardado automático de resultados.

La aplicación se arranca con un solo comando desde la raíz del proyecto:



La interfaz debe mostrar:

Componente de la interfaz	Descripción	Obligatorio
Sidebar con ejemplos	3 textos de ejemplo cargables con un clic	Sí
Área de texto de entrada	Campo para introducir el texto a analizar	Sí

Botón Analizar	Lanza los 5 análisis NLP	Sí
Tab Sentimiento	Badge de color + barras de puntuación y confianza	Sí
Tab Entidades (NER)	Grid de entidades por categoría con iconos	Sí
Tab Intención	Intención, subcategoría, urgencia, acción sugerida	Sí
Tab Resumen	3 expanders: ultracorto, medio, detallado	Sí
Tab Clasificación	Tema, tipo, canal y barra de prioridad	Sí
Barra de estado / guardado	Confirma guardado automático con ruta del archivo	Sí
Historial (opcional)	Lista de análisis previos guardados	No

☐ El guardado automático debe activarse tras cada análisis exitoso. La barra de estado debe mostrar: '✓ Análisis completado y guardado automáticamente' con la ruta del archivo generado.

## 7.1 — Diseña el flujo de la aplicación Streamlit

Describe con tus propias palabras el flujo de ejecución de la app cuando el usuario pulsa el botón Analizar. ¿Qué funciones se llaman? ¿En qué orden? ¿Qué se muestra?

## 7.1 — Diseña el flujo de la aplicación Streamlit

Flujo de ejecución cuando el usuario pulsa "Analizar"



Funciones involucradas en orden		
Paso	Función	Archivo
1	<code>st.text_area()</code>	interface/demo_app.py
2	<code>st.form_submit_button()</code>	interface/demo_app.py
3	<code>validar_texto()</code>	src/utlis.py
4	<code>analizar_texto()</code>	src/analizador.py
5	<code>_ analisis_individual()</code>	src/niveles.py
6	<code>limpiar_respuesta_json()</code>	src/utlis.py
7	<code>guardar_resultado()</code>	almacenamiento/guardar.py

Qué se muestra	
Elemento	Descripción
Status card	"Analizando..." con progress spinner
Tiempo	Duración del análisis (~3-5 seg)
5 Cards	Una para cada tipo de análisis
Mensaje éxito	"Análisis completado en XXX seg"
Descargar	Botones JSON y TXT (si hay resultados)

## 7.2 — Diferencias entre ejecución CLI y Streamlit

Aspecto	Ejecución CLI (demo_nlp.py)	Interfaz Streamlit (demo_app.py)
---------	-----------------------------	----------------------------------

## 7.2 — Diferencias entre ejecución CLI y Streamlit

Aspecto	CLI (main.py)	Interfaz Streamlit (demo_app.py)
Entrada de texto	<code>sys.argv</code> o <code>input()</code>	<code>st.text_area()</code>
Salida de resultados	<code>print()</code> en terminal	<code>st.tabs()</code> + cards con markdown
Re-ejecución	Nuevo comando cada vez	Auto-refresh en Streamlit
Estado de la app	Efímero (termina al acabar)	Persistente (mantiene estado)
Guardado	Con flag <code>--guardar</code>	Checkbox en sidebar
Interfaz usuario	Texto plano	Visual/gráfica
Feedback visual	Solo texto	Spinners, progress, success messages
Descarga archivos	No automático	Botones de download
Historial	Con flag <code>--listar</code>	Ver en carpeta resultados/

### Comparación detallada

MODULO CLI

```
$ python main.py "texto a analizar"
```

CLI Mode

Texto a analizar: [el usuario escribe]

Analizando ...

---

{ JSON output en terminal }

MODULO STREAMLIT

http://localhost:8501

[=====TEXTO=====]

[=====ANALIZAR=====] (botón rojo ●)

SENTIMIENTO  
😊

ENTIDADES  
📌

INTENCIÓN  
🎯

+ Cards visuales

RESUMEN  
📄

CLASIFIC.  
📊

## Cuándo usar cada uno

Modo	Cuándo usar
CLI	Scripts automatizados, batch processing, servidores sin GUI
Streamlit	Demo, usuarios finales, exploración interactiva

## Resumen

- **CLI:** Rápido, automatizable, sin interfaz gráfica
- **Streamlit:** Visual, interactivo, mejor UX, guarda automáticamente

Ambos usan las mismas funciones de análisis (`src/niveles.py`), solo cambia la capa de presentación.

## BLOQUE 8 — Reflexión Integrada

□ *Reflexión final:* Describe el flujo profesional completo que acabas de implementar. ¿Qué aporta cada paso (análisis → modularización → tests → CI/CD → almacenamiento → interfaz Streamlit → docs)? ¿Qué hubiera pasado si te saltas alguno?

## 8.1 — Flujo profesional completo

Describe con tus palabras qué aprendiste en cada fase y por qué el orden importa:

## Qué aprendí?

A lo largo de este proyecto he aprendido a transformar un script monolítico de NLP en un sistema profesional y modular. He aplicado principios de diseño como SRP (Responsabilidad Única), inyección de dependencias y separación de capas.

## Principales desafíos

- **Modularización del código heredado:** El código original (`InicialNLP.py`) mezclaba toda la lógica en un solo archivo. Separarlo en módulos independientes (`cliente.py`, `niveles.py`, `analizador.py`) fue un desafío conceptual y práctico.
- **Tests sin dependencias externas:** Usar mocks para no depender del modelo real de Ollama durante los tests.
- **Pipeline CI/CD:** Configurar GitHub Actions para que valide el código automáticamente en cada push.
- **Interfaz Streamlit:** Crear una interfaz web profesional que fuera intuitiva y visualmente atractiva.

## 8.2 — ¿Cómo usaste la IA como copiloto?

Para cada bloque, indica en qué momento pediste ayuda a la IA y cómo validaste su respuesta antes de aplicarla:

Bloque	¿Para qué usé la IA?	¿Cómo verifiqué su respuesta?
1	Mezcla de responsabilidades	Separación en módulos: src/, almacenamiento/, interface/
2	Entrypoint coupled	CLI (main.py) + Web (Streamlit) + APIready
3	Cliente hardcodeado	get_client() con detección automática de Ollama
4	Sin contratos	Tipado con docstrings y validación en utils.py
5	Sin persistencia	almacenamiento/guardar.py con timestamps únicos
6	Sin manejo de errores	Try/except en funciones críticas
7	Sin tests	24 tests unitarios con pytest
8	Configuración débil	.env + dotenv
9	Archivo único	9 archivos Python organizados
10	Solo CLI	Interfaz Streamlit profesional

## 8.3 — Autoevaluación

Marca con una X tu nivel de confianza al finalizar el ejercicio:

Nivel	Descripción	¿Qué necesitarías repasar?
<input type="checkbox"/> 1	Necesito más práctica	
<input type="checkbox"/> 2	Lo entiendo con ayuda	
<input checked="" type="checkbox"/> 3	<b>Lo aplico con seguridad</b>	
<input type="checkbox"/> 4	Podría enseñarlo	

## BLOQUE 9 — Reflexión MemoriaFinal.md

En la carpeta reflexiones/, crea un archivo llamado memoriaFinal.md donde explicaréis el porqué el código original está mal configurado y qué cambios realizasteis. Incluye también la decisión de usar Streamlit en lugar de Tkinter.

### 9.1 — Estructura sugerida del memoriaFinal.md

```
Memoria Final – Refactorización NLP con Streamlit

1. Problemas detectados en el código heredado
- ...

2. Decisiones de diseño tomadas
- Separación en módulos (SRP)
- Elección de Streamlit sobre Tkinter: ...

3. Cambios realizados por bloque
- Bloque 1 – Análisis: ...
- Bloque 2 – Modularización: ...
- Bloque 3 – Tests: ...
- Bloque 4 – CI/CD: ...
- Bloque 5 – Almacenamiento: ...
- Bloque 6 – Docs: ...
- Bloque 7 – Streamlit: ...

4. Lecciones aprendidas
- ...

5. ¿Qué mejorarías con más tiempo?
- ...
```

Eh dejado la memoria final en D:\ADEV\Proyectos\VScode\NLP\_Streamlit\reflexiones

### 9.2 — Escribe aquí tu reflexión (borrador)

Este proyecto demuestra la transformación de un script Python básico en un sistema profesional con:

- **Product-ready:** Persistencia, manejo de errores, configuración
- **Interfaz profesional:** Streamlit lista para demo empresarial
- **Calidad de código:** Tests, linting, tipo check
- **Arquitectura limpia:** Separación de capas (UI, lógica, datos)

La modularización no solo es buena práctica, sino que permite escalar, testear y mantener el código de forma profesional.



## Interfaz Streamlit con NLP



---

Fundación Dicampus · Inclusión Socio-Laboral · Módulo 4 · Código Heredado